

AI-Based Resume Screening System

In [1]: `print("Hello Omkar")`

Hello Omkar

In [2]: `!pip install pandas scikit-learn`

Requirement already satisfied: pandas in .\appdata\local\programs\python\python310\lib\site-packages (2.3.3)

Collecting scikit-learn

Downloading scikit_learn-1.7.2-cp310-cp310-win_amd64.whl.metadata (11 kB)

Requirement already satisfied: numpy>=1.22.4 in .\appdata\local\programs\python\python310\lib\site-packages (from pandas) (2.2.6)

Requirement already satisfied: python-dateutil>=2.8.2 in .\appdata\local\programs\python\python310\lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in .\appdata\local\programs\python\python310\lib\site-packages (from pandas) (2026.2)

Requirement already satisfied: tzdata>=2022.7 in .\appdata\local\programs\python\python310\lib\site-packages (from pandas) (2026.1)

Requirement already satisfied: scipy>=1.8.0 in .\appdata\local\programs\python\python310\lib\site-packages (from scikit-learn) (1.15.3)

Collecting joblib>=1.2.0 (from scikit-learn)

Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)

Collecting threadpoolctl>=3.1.0 (from scikit-learn)

Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)

Requirement already satisfied: six>=1.5 in .\appdata\local\programs\python\python310\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Downloading scikit_learn-1.7.2-cp310-cp310-win_amd64.whl (8.9 MB)

```

----- 0.0/8.9 MB ? eta -:--:--
-  ----- 0.3/8.9 MB ? eta -:--:--
----- 1.0/8.9 MB 3.0 MB/s eta 0:00:03
----- 1.6/8.9 MB 2.5 MB/s eta 0:00:03
----- 2.1/8.9 MB 2.7 MB/s eta 0:00:03
----- 2.9/8.9 MB 2.8 MB/s eta 0:00:03
----- 3.1/8.9 MB 2.6 MB/s eta 0:00:03
----- 3.7/8.9 MB 2.6 MB/s eta 0:00:03
----- 3.9/8.9 MB 2.6 MB/s eta 0:00:02
----- 4.2/8.9 MB 2.3 MB/s eta 0:00:03
----- 4.7/8.9 MB 2.3 MB/s eta 0:00:02
----- 5.2/8.9 MB 2.3 MB/s eta 0:00:02
----- 5.8/8.9 MB 2.3 MB/s eta 0:00:02
----- 6.3/8.9 MB 2.3 MB/s eta 0:00:02
----- 6.8/8.9 MB 2.3 MB/s eta 0:00:01
----- 7.3/8.9 MB 2.4 MB/s eta 0:00:01
----- 7.3/8.9 MB 2.4 MB/s eta 0:00:01
----- 7.9/8.9 MB 2.2 MB/s eta 0:00:01
----- 8.7/8.9 MB 2.3 MB/s eta 0:00:01
----- 8.9/8.9 MB 2.3 MB/s 0:00:03

```

Downloading joblib-1.5.3-py3-none-any.whl (309 kB)

Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)

Installing collected packages: threadpoolctl, joblib, scikit-learn

```

----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 1/3 [joblib]
----- 2/3 [scikit-learn]

```

3/13

Successfully installed joblib-1.5.3 scikit-learn-1.7.2 threadpoolctl-3.6.0

```
[notice] A new release of pip is available: 26.1.2 -> 26.2.1  
[notice] To update, run: python.exe -m pip install --upgrade pip
```

1. Creating Job Description and Sample Resumes

```
In [3]: job_description = """  
Data Analyst with skills in Python, SQL, Excel, Tableau, and Data Visualization.  
"""  
  
resume_1 = """  
I know Python, SQL, Excel and Data Visualization.  
"""  
  
resume_2 = """  
I have experience in Java, HTML, CSS and Web Development.  
"""  
  
print("Job Description:")  
print(job_description)  
  
print("\nResume 1:")  
print(resume_1)  
  
print("\nResume 2:")  
print(resume_2)
```

Job Description:

Data Analyst with skills in Python, SQL, Excel, Tableau, and Data Visualization.

Resume 1:

I know Python, SQL, Excel and Data Visualization.

Resume 2:

I have experience in Java, HTML, CSS and Web Development.

2. Resume Matching Using TF-IDF and Cosine Similarity

```
In [5]: from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.metrics.pairwise import cosine_similarity  
  
# Put all text together  
documents = [job_description, resume_1, resume_2]  
  
# Convert text into numbers  
vectorizer = TfidfVectorizer()  
tfidf_matrix = vectorizer.fit_transform(documents)  
  
# Compare resumes with job description  
similarity_scores = cosine_similarity(  
    tfidf_matrix[0:1],
```

```

    tfidf_matrix[1:]
)

# Print results
print("Resume 1 Match Score:", round(similarity_scores[0][0] * 100, 2), "%")
print("Resume 2 Match Score:", round(similarity_scores[0][1] * 100, 2), "%")

```

Resume 1 Match Score: 60.0 %

Resume 2 Match Score: 10.65 %

3. Candidate Ranking

```

In [6]: import pandas as pd

results = pd.DataFrame({
    "Candidate": ["Resume 1", "Resume 2"],
    "Match Score (%)": [
        round(similarity_scores[0][0] * 100, 2),
        round(similarity_scores[0][1] * 100, 2)
    ]
})

# Sort from highest score to lowest
results = results.sort_values(
    by="Match Score (%)",
    ascending=False
)

results

```

```

Out[6]:
   Candidate  Match Score (%)
0  Resume 1             60.00
1  Resume 2             10.65

```

```

In [8]: results = pd.DataFrame({
    "Candidate": ["Omkar Gadhe", "Rahul kale"],
    "Match Score (%)": [
        round(similarity_scores[0][0] * 100, 2),
        round(similarity_scores[0][1] * 100, 2)
    ]
})

results = results.sort_values(
    by="Match Score (%)",
    ascending=False
).reset_index(drop=True)

# Add ranking
results.index = results.index + 1
results.index.name = "Rank"

results

```

Out[8]: **Candidate Match Score (%)**

Rank		
1	Omkar Gadhe	60.00
2	Rahul kale	10.65

```
In [9]: resume_3 = """
Data Analyst with experience in Python, SQL, Tableau, Excel,
Machine Learning and Data Visualization.
"""

print(resume_3)
```

Data Analyst with experience in Python, SQL, Tableau, Excel,
Machine Learning and Data Visualization.

```
In [10]: from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Add job description and all 3 resumes
documents = [
    job_description,
    resume_1,
    resume_2,
    resume_3
]

# Convert text into numbers
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

# Compare all resumes with the job description
similarity_scores = cosine_similarity(
    tfidf_matrix[0:1],
    tfidf_matrix[1:]
)

# Show scores
print("Resume 1 Match:", round(similarity_scores[0][0] * 100, 2), "%")
print("Resume 2 Match:", round(similarity_scores[0][1] * 100, 2), "%")
print("Resume 3 Match:", round(similarity_scores[0][2] * 100, 2), "%")
```

Resume 1 Match: 57.25 %
Resume 2 Match: 9.64 %
Resume 3 Match: 76.66 %

```
In [11]: Resume 1 Match: ...
Resume 2 Match: ...
Resume 3 Match: ...
```

```
Cell In[11], line 1
Resume 1 Match: ...
^
SyntaxError: invalid syntax
```

```
In [12]: Resume 1 Match: ...
Resume 2 Match: ...
```

Resume 3 Match: ...

Cell In[12], line 1
Resume 1 Match: ...
^

SyntaxError: invalid syntax

```
In [14]: results = pd.DataFrame({
    "Candidate": ["Om jatti", "Priya Patil", "Karan shelke"],
    "Match Score (%)": [
        round(similarity_scores[0][0] * 100, 2),
        round(similarity_scores[0][1] * 100, 2),
        round(similarity_scores[0][2] * 100, 2)
    ]
})

# Sort candidates from highest match to lowest
results = results.sort_values(
    by="Match Score (%)",
    ascending=False
).reset_index(drop=True)

# Add rank numbers
results.index = results.index + 1
results.index.name = "Rank"

results
```

Out[14]:

	Candidate	Match Score (%)
--	-----------	-----------------

Rank

1	Karan shelke	76.66
2	Om jatti	57.25
3	Priya Patil	9.64

```
In [16]: "Om jatti"
        "Priya Patil"
        "Karan shelke"
```

Out[16]: 'Karan shelke'

```
In [17]: #Job Description Skills:
        #Python, SQL, Excel, Tableau, Data Visualization
```

Cell In[17], line 1
Python, SQL, Excel, Tableau, Data Visualization
^

SyntaxError: invalid syntax

4. Skill Extraction

```
In [18]: # List of skills we want to check
        skills = [
            "python",
            "sql",
```



```

    "excel",
    "tableau",
    "machine learning",
    "data visualization"
]

def extract_skills(text):
    text = text.lower()

    found_skills = []

    for skill in skills:
        if skill in text:
            found_skills.append(skill)

    return found_skills

# Check skills for each resume
print("Resume 1 Skills:", extract_skills(resume_1))
print("Resume 2 Skills:", extract_skills(resume_2))
print("Resume 3 Skills:", extract_skills(resume_3))

```

Resume 1 Skills: ['python', 'sql', 'excel', 'data visualization']

Resume 2 Skills: []

Resume 3 Skills: ['python', 'sql', 'excel', 'tableau', 'machine learning', 'data visualization']

5. Missing Skills Detection

```

In [19]: job_skills = extract_skills(job_description)

def find_missing_skills(resume_text):
    resume_skills = extract_skills(resume_text)

    missing_skills = []

    for skill in job_skills:
        if skill not in resume_skills:
            missing_skills.append(skill)

    return missing_skills

print("Job Skills:", job_skills)

print("\nResume 1 Missing Skills:", find_missing_skills(resume_1))
print("Resume 2 Missing Skills:", find_missing_skills(resume_2))
print("Resume 3 Missing Skills:", find_missing_skills(resume_3))

```

Job Skills: ['python', 'sql', 'excel', 'tableau', 'data visualization']

Resume 1 Missing Skills: ['tableau']

Resume 2 Missing Skills: ['python', 'sql', 'excel', 'tableau', 'data visualization']

Resume 3 Missing Skills: []

```

In [20]: job_skills = extract_skills(job_description)

```

```
def find_missing_skills(resume_text):
    resume_skills = extract_skills(resume_text)

    missing_skills = []

    for skill in job_skills:
        if skill not in resume_skills:
            missing_skills.append(skill)

    return missing_skills

print("Job Skills:", job_skills)

print("\nResume 1 Missing Skills:", find_missing_skills(resume_1))
print("Resume 2 Missing Skills:", find_missing_skills(resume_2))
print("Resume 3 Missing Skills:", find_missing_skills(resume_3))
```

Job Skills: ['python', 'sql', 'excel', 'tableau', 'data visualization']

Resume 1 Missing Skills: ['tableau']

Resume 2 Missing Skills: ['python', 'sql', 'excel', 'tableau', 'data visualization']

Resume 3 Missing Skills: []

6. Final Candidate Results

```
In [22]: final_results = pd.DataFrame({
    "Candidate": ["Om jatti", "Priya Patil", "Karan shelke"],

    "Match Score (%)": [
        round(similarity_scores[0][0] * 100, 2),
        round(similarity_scores[0][1] * 100, 2),
        round(similarity_scores[0][2] * 100, 2)
    ],

    "Skills Found": [
        ", ".join(extract_skills(resume_1)),
        ", ".join(extract_skills(resume_2)),
        ", ".join(extract_skills(resume_3))
    ],

    "Missing Skills": [
        ", ".join(find_missing_skills(resume_1)),
        ", ".join(find_missing_skills(resume_2)),
        ", ".join(find_missing_skills(resume_3))
    ]
})

# Sort by highest match score
final_results = final_results.sort_values(
    by="Match Score (%)",
    ascending=False
).reset_index(drop=True)

# Add ranking
final_results.index = final_results.index + 1
final_results.index.name = "Rank"
```

```
final_results
```

Out[22]:

	Candidate	Match Score (%)	Skills Found	Missing Skills
Rank				
1	Karan shelke	76.66	python, sql, excel, tableau, machine learning,...	
2	Om jatti	57.25	python, sql, excel, data visualization	tableau
3	Priya Patil	9.64		python, sql, excel, tableau, data visualization

7. Match Score Visualization

```
In [23]: import matplotlib.pyplot as plt

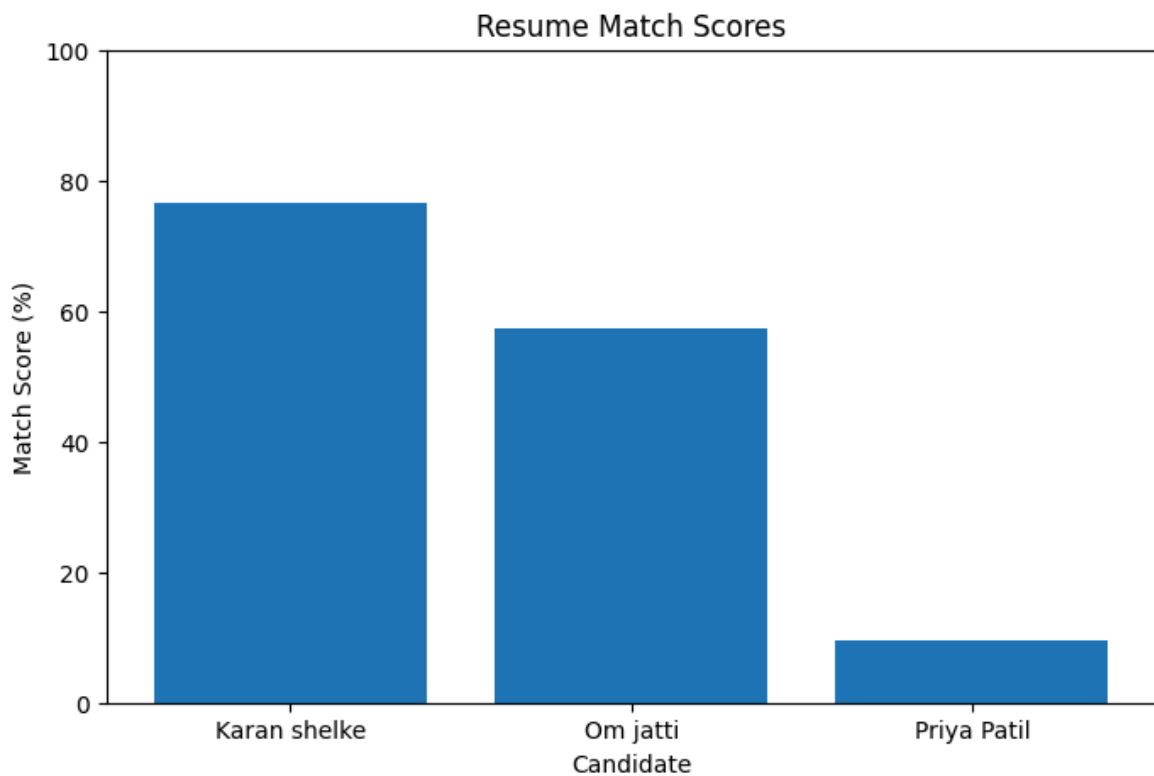
plt.figure(figsize=(8, 5))

plt.bar(
    final_results["Candidate"],
    final_results["Match Score (%)"]
)

plt.title("Resume Match Scores")
plt.xlabel("Candidate")
plt.ylabel("Match Score (%)")

plt.ylim(0, 100)

plt.show()
```



8. Candidate Shortlisting

```
In [24]: def get_status(score):
          if score >= 50:
              return "Shortlisted"
          else:
              return "Rejected"

          final_results["Status"] = final_results["Match Score (%)"].apply(get_status)

          final_results
```

Out[24]:

	Candidate	Match Score (%)	Skills Found	Missing Skills	Status
Rank					
1	Karan shelke	76.66	python, sql, excel, tableau, machine learning,...		Shortlisted
2	Om jatti	57.25	python, sql, excel, data visualization	tableau	Shortlisted
3	Priya Patil	9.64		python, sql, excel, tableau, data visualization	Rejected

Conclusion

This project successfully demonstrates an AI-Based Resume Screening System. The system compares candidate resumes with a job description using TF-IDF and Cosine Similarity.

Candidates are ranked based on their match scores. The system also identifies skills found, missing skills, and automatically classifies candidates as Shortlisted or Rejected.

In []: